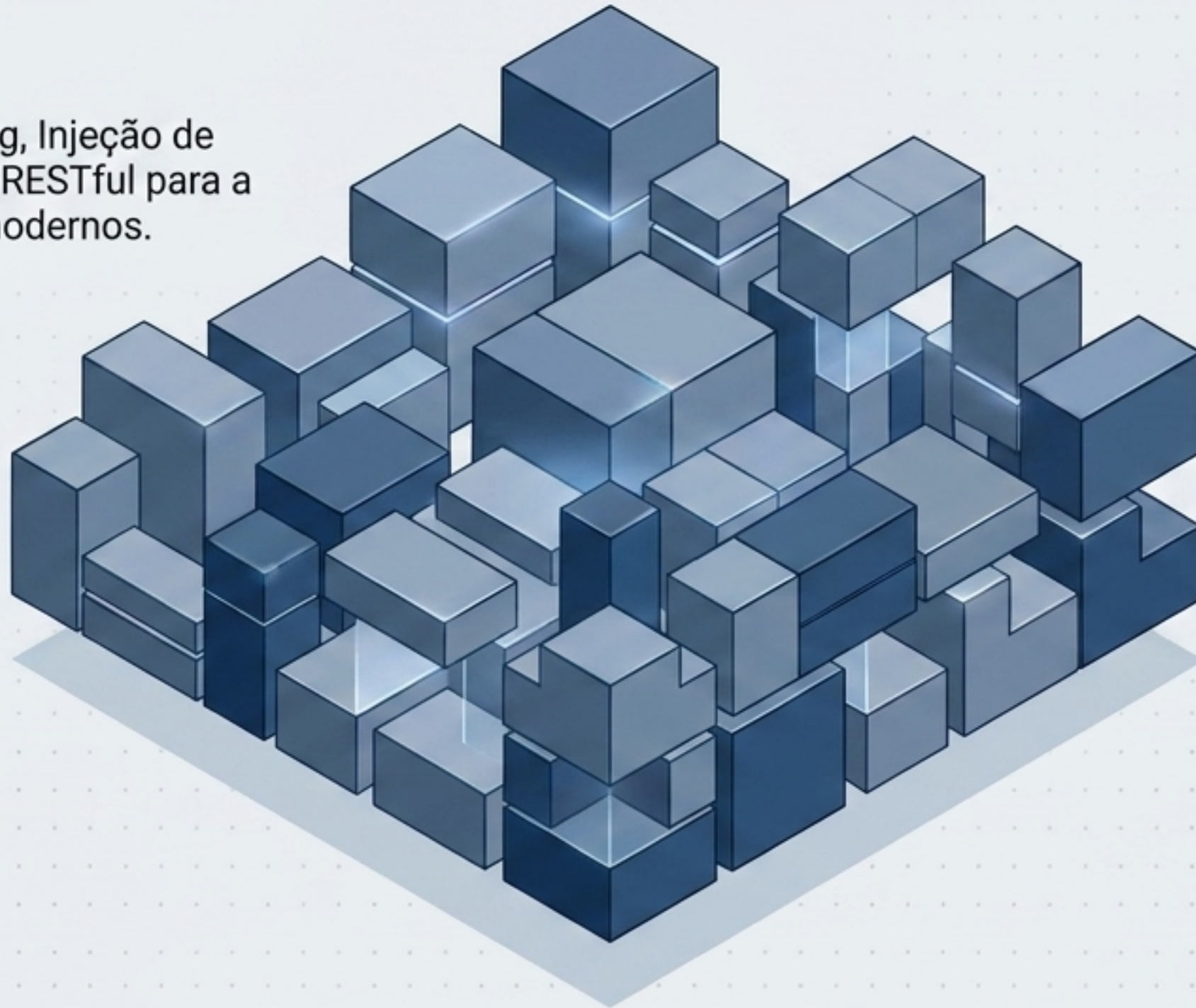


A Arquitetura de Dentro para Fora: O Projeto Digital Modular

POS TECH - Engenharia de Software

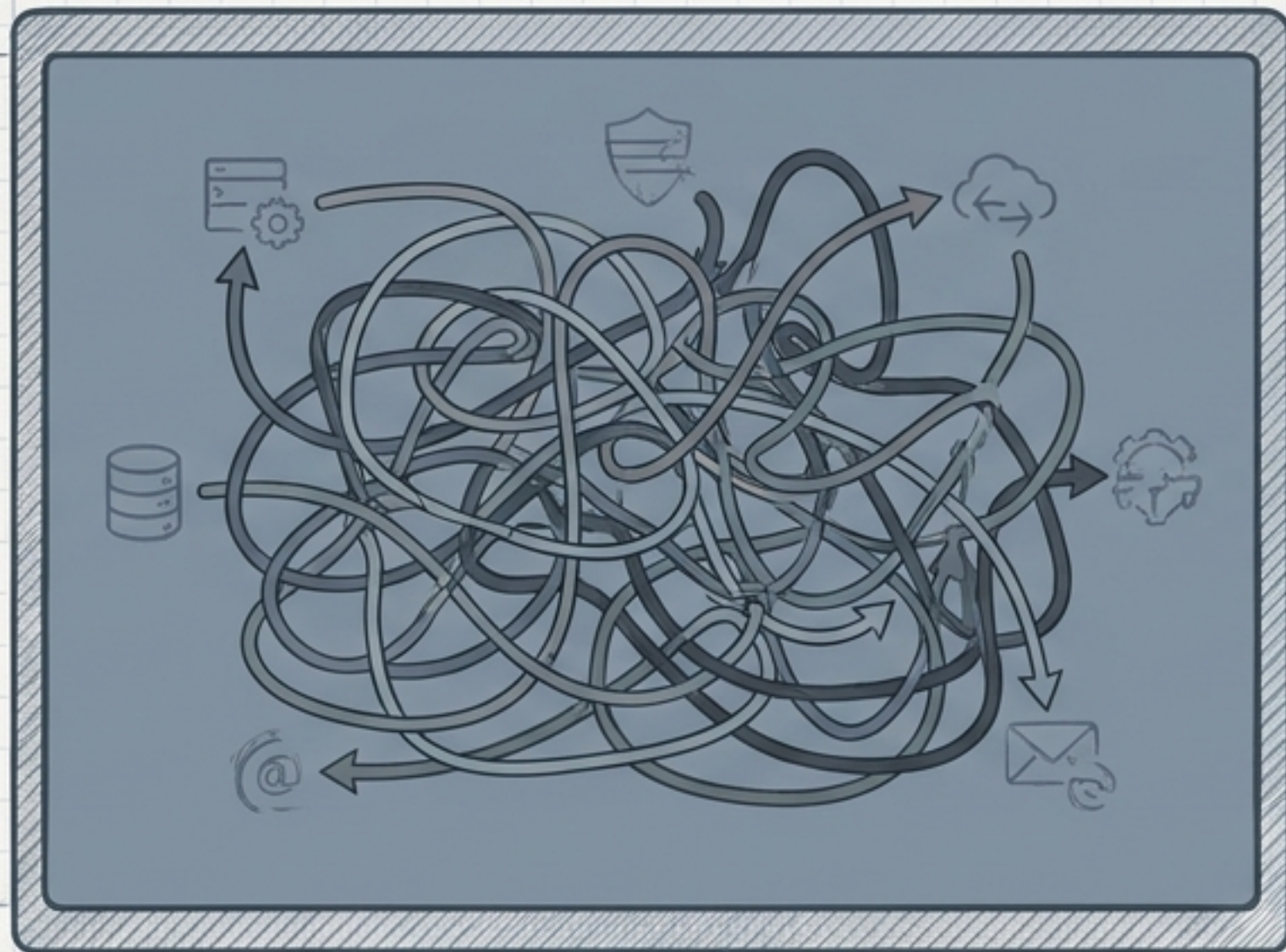
Dominando o ecossistema Spring, Injeção de Dependência, Build Tools e APIs RESTful para a construção de microsserviços modernos.



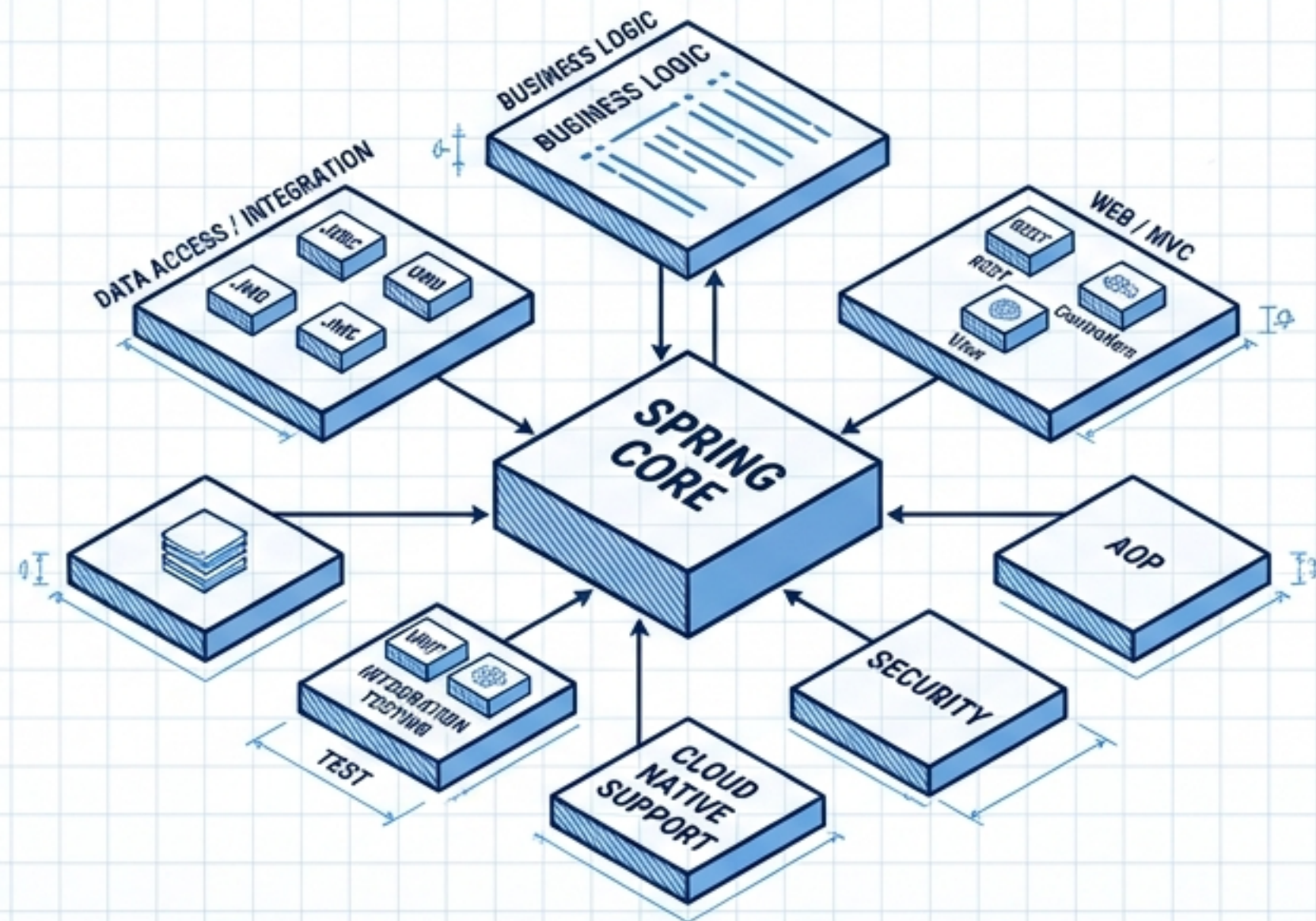
O Passado: Containers EJB

← BEFORE -- AFTER →

O Presente: Spring Framework



Complexidade de infraestrutura misturada e acoplada diretamente à lógica de negócio.



Foco exclusivo nas regras de negócio. A infraestrutura pesada é abstraída e gerenciada pelo framework.

Lançado em 2003, o Spring revolucionou o ecossistema Java ao substituir a rigidez dos monolitos pela agilidade da arquitetura modular.



Spring Boot

0 motor de arranque. Elimina configurações complexas, trazendo servidores embutidos e autoconfiguração.



Spring Data

0 arquivista. Abstração unificada para acesso a bancos de dados, mapeamento objeto-relacional automático e paginação.



Spring Framework (Core)



Spring Security

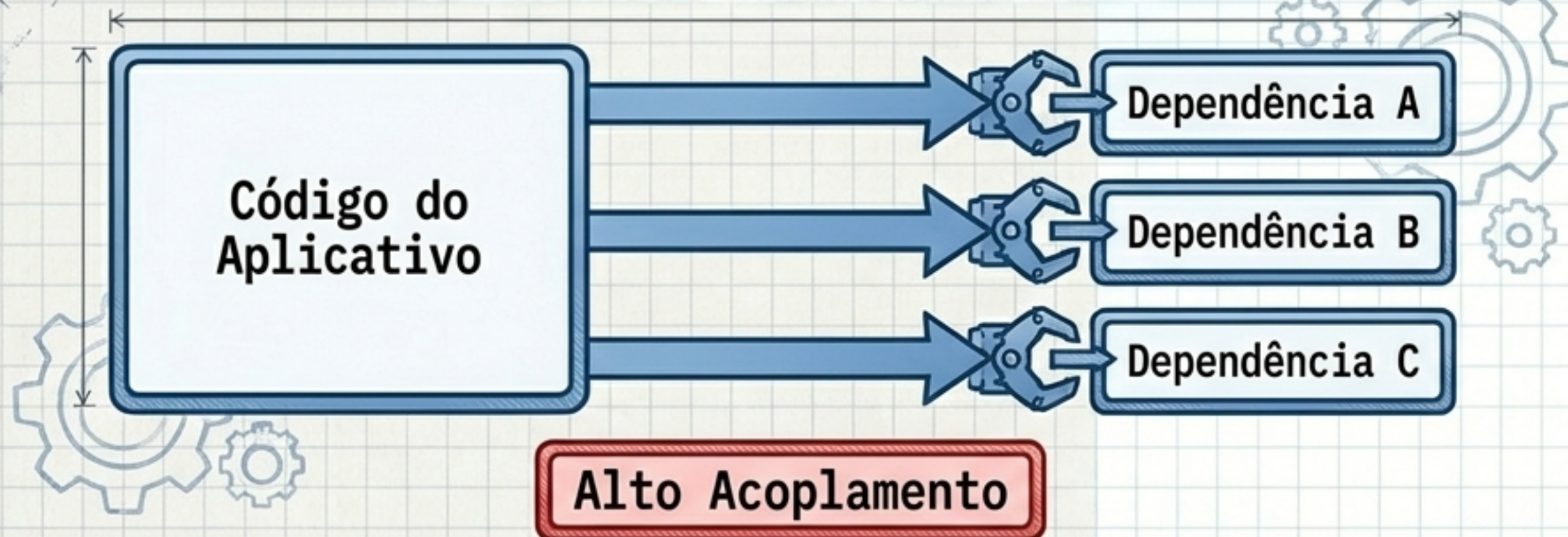
0 escudo. Prevenção nativa contra ataques (XSS/CSRF), controle de acesso a URLs, tokens e autenticação.



Spring Cloud

0 orquestrador. Gerenciamento de configuração, descoberta de serviços, balanceamento de carga e circuit breakers.

Fluxo Procedural Tradicional



Inversão de Controle - IoC



Insights Chave

- O Princípio de Hollywood:
"Não nos chame, nós chamamos chamamos você." O fluxo de controle é invertido.
- Vantagens da IoC:
 - Alta Modularidade
 - Baixo Acoplamento
 - Facilidade de Testabilidade

A Configuração

```
@Configuration
public class AppConfig {

    @Bean
    public CustomerRepository
    customerRepository() {
        return new CustomerRepository();
    }
}
```

O Controlador

```
public class OrderController {

    private final CustomerRepository
    repository;

    public OrderController(
    CustomerRepository repository) {
        this.repository = repository;
    }
}
```



A Mágica da Injeção de Dependência (DI):

O objeto é instanciado e injetado automaticamente como dependência pelo Spring Container, eliminando a necessidade de usar 'new' dentro do controlador.

Princípio da Inversão de Dependência (DIP)

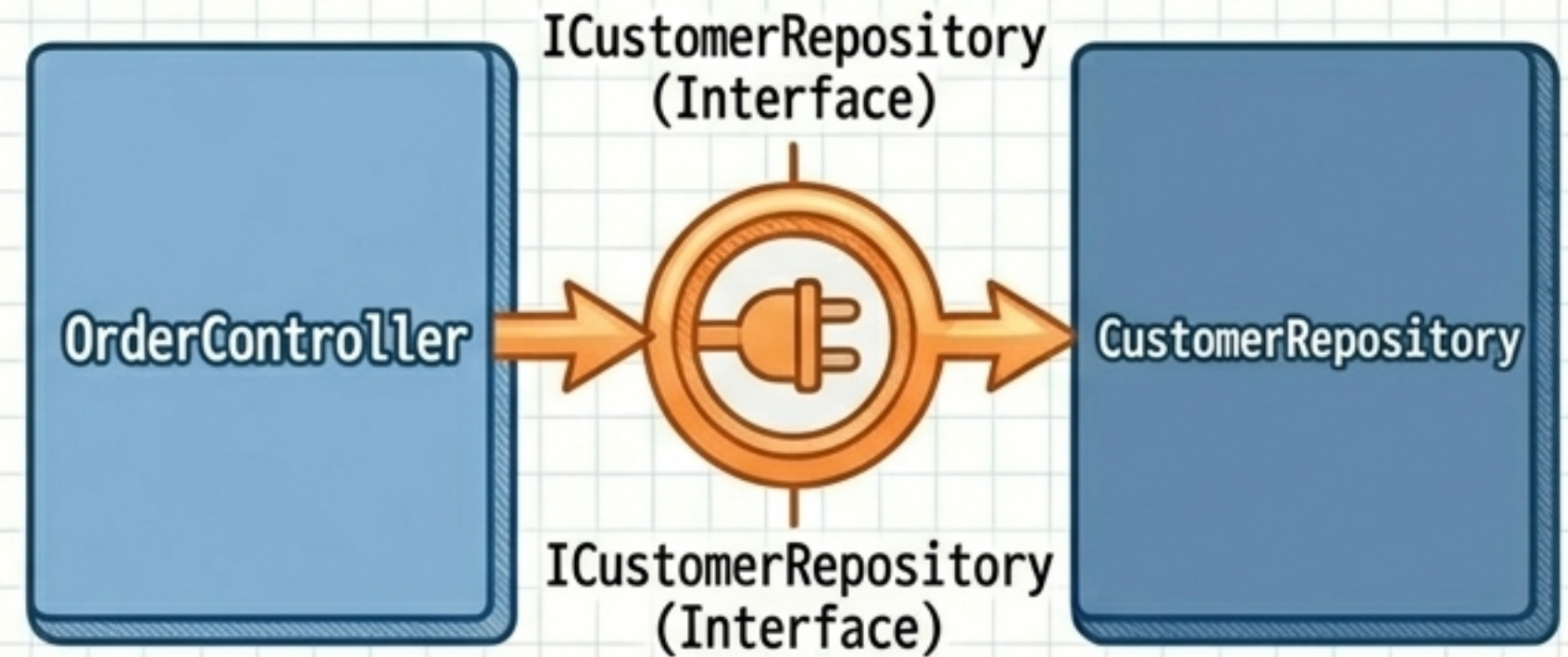
Alto Acoplamento

Dependência de implementações concretas.



Desacoplamento via DIP

Dependência de abstrações.



Abstração: Depender de interfaces, não de implementações concretas.



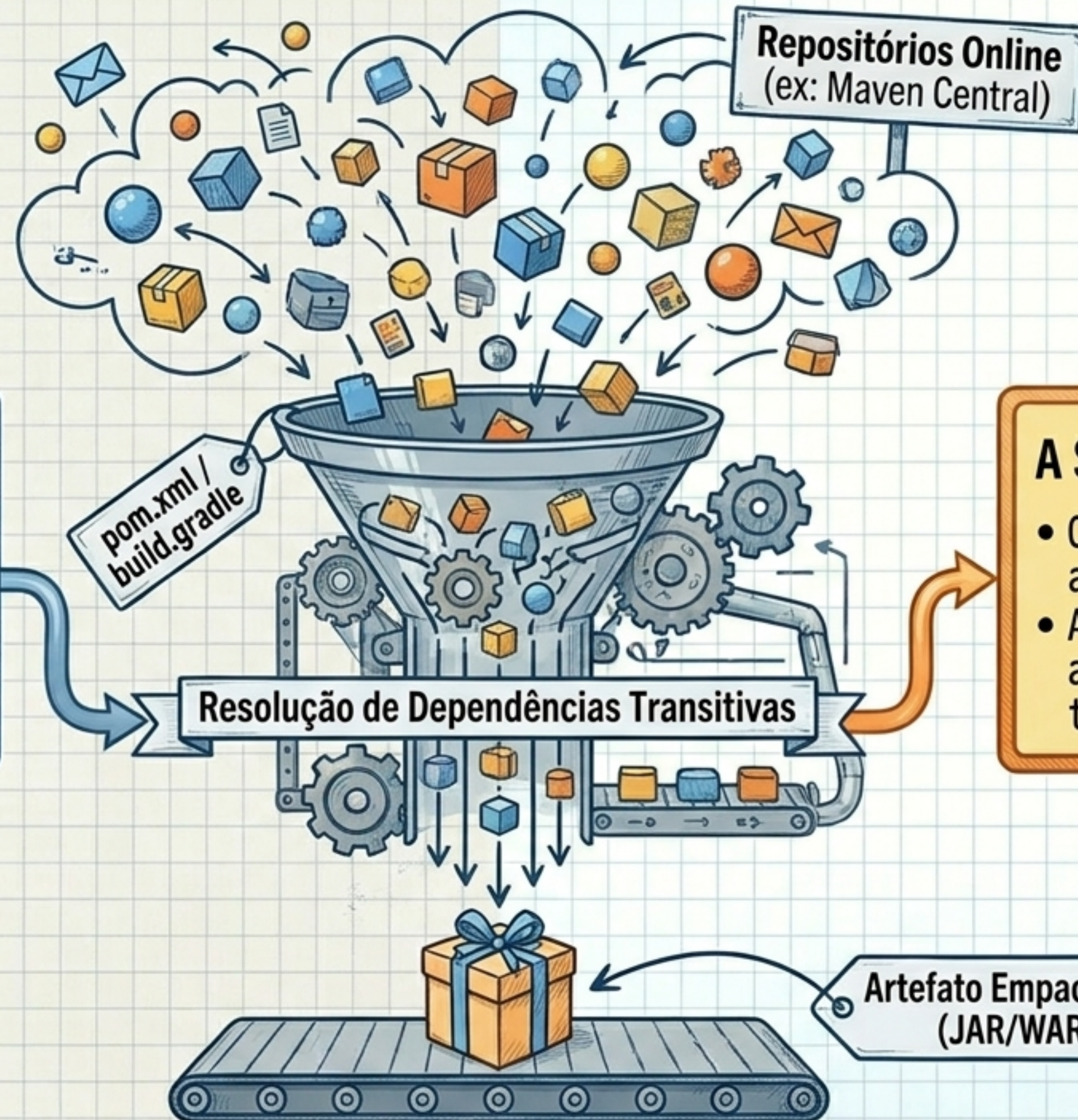
Testabilidade: Permite a troca fácil de componentes, como a inserção de Mocks para testes.



O Trio Moderno: IoC, DI e DIP trabalham em harmonia para construir sistemas resilientes.

O Problema Antigo:

- Download manual de arquivos .jar
- Configuração complexa de classpath
- Conflitos de versão de bibliotecas



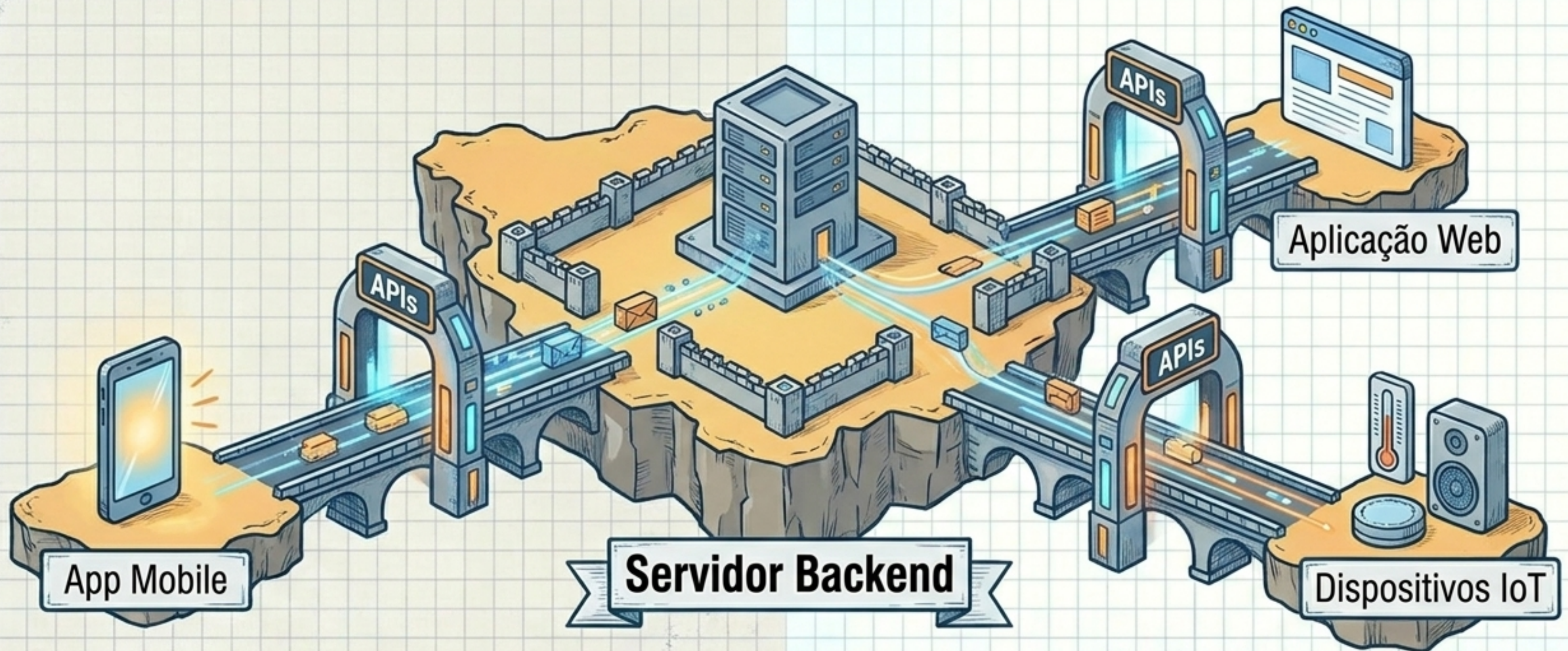
A Solução Automação:

- O arquivo central dita as regras do projeto.
- A ferramenta resolve automaticamente a teia de dependências.

O Duelo das Ferramentas de Build

Atributo		Apache Maven	Gradle
	Linguagem de Configuração	• Baseado em XML estruturado.	• DSL expressiva baseada em Groovy ou Kotlin.
	Estrutura e Flexibilidade	• Altamente rígido, com conjunto fixo de plugins e convenções.	• Altamente flexível, projetado para modularização e personalização profunda.
	Desempenho	• Velocidade padrão de compilação linear.	• Execução incremental inteligente e super-rápida (reaproveita builds anteriores).
	Caso de Uso Ideal	• Aplicações padronizadas; pioneiro e amplamente compreendido.	• Projetos complexos, grandes monolitos divididos ou ecossistemas modernos.

O que são APIs? A fundação da comunicação entre sistemas.



Comunicação **fluida e padronizada** entre partes.

Totalmente **independente** da linguagem de programação.

Garante a **interoperabilidade** entre plataformas heterogêneas.

O Espectro de Protocolos e APIs



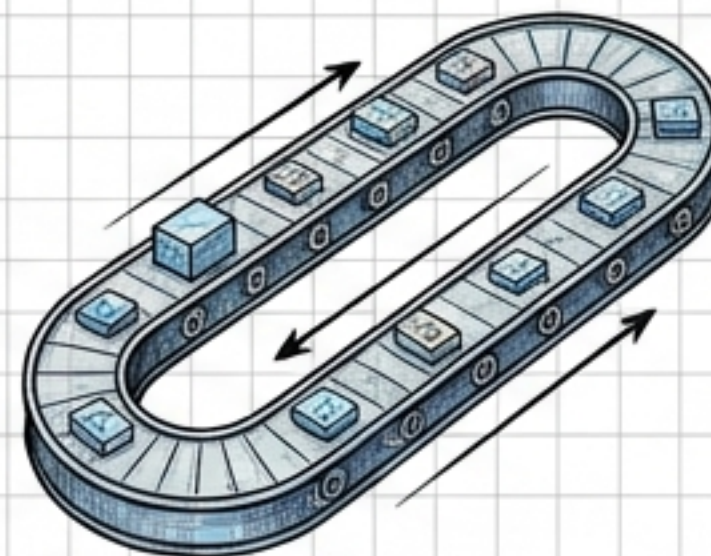
SOAP

Baseado em XML, pesado e com alto rigor de especificações. Contratos estritos.



RPC (Remote Procedure Call)

Forte acoplamento focado na execução de ações diretas no servidor remoto.



Websocket

Conexão aberta, persistente e bidirecional. Ideal para comunicação em tempo real.

Padrão de Mercado



REST (Representational State Transfer)

O padrão moderno. Arquitetura leve baseada nativamente no protocolo HTTP e totalmente stateless.

Arquitetura RESTful

Pillar 1

Pillar 4



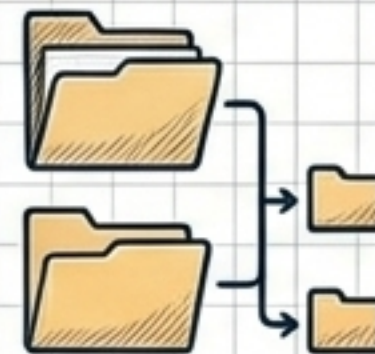
Simplicidade HTTP

Uso direto e semântico dos métodos nativos do protocolo: GET, POST, PUT, DELETE.



Escalabilidade Independente

Clara separação entre cliente e servidor. A natureza 'Stateless' (sem estado) permite escalar a infraestrutura horizontalmente.



Baseada em Recursos

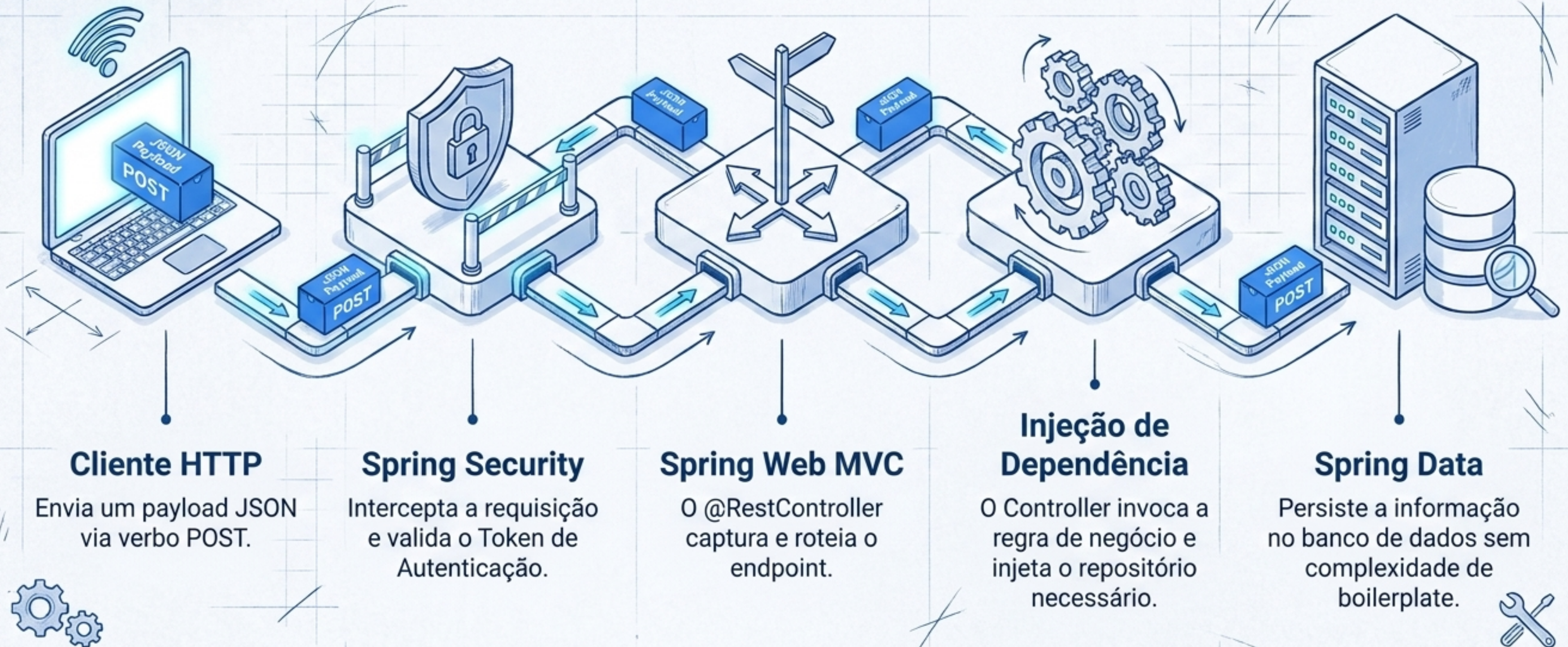
As URLs são organizadas de forma lógica e hierárquica, refletindo os dados e entidades reais do domínio.



Segurança Nativa

Integração direta com o protocolo HTTPS e cabeçalhos de autorização padrão, como tokens JWT e OAuth.

A Jornada de uma Requisição REST



Um Ecossistema Expansivo para Desafios Empresariais



Spring Batch

Processamento otimizado de operações em lote de alto volume de dados.



Spring Cloud Data Flow

Orquestração de microsserviços e pipelines de dados componíveis em tempo real.



Spring Authorization Server

Provedor nativo e personalizável para identidades OpenID e OAuth2.



Spring for GraphQL

Integrações modernas para a construção de APIs flexíveis baseadas em consultas gráficas.

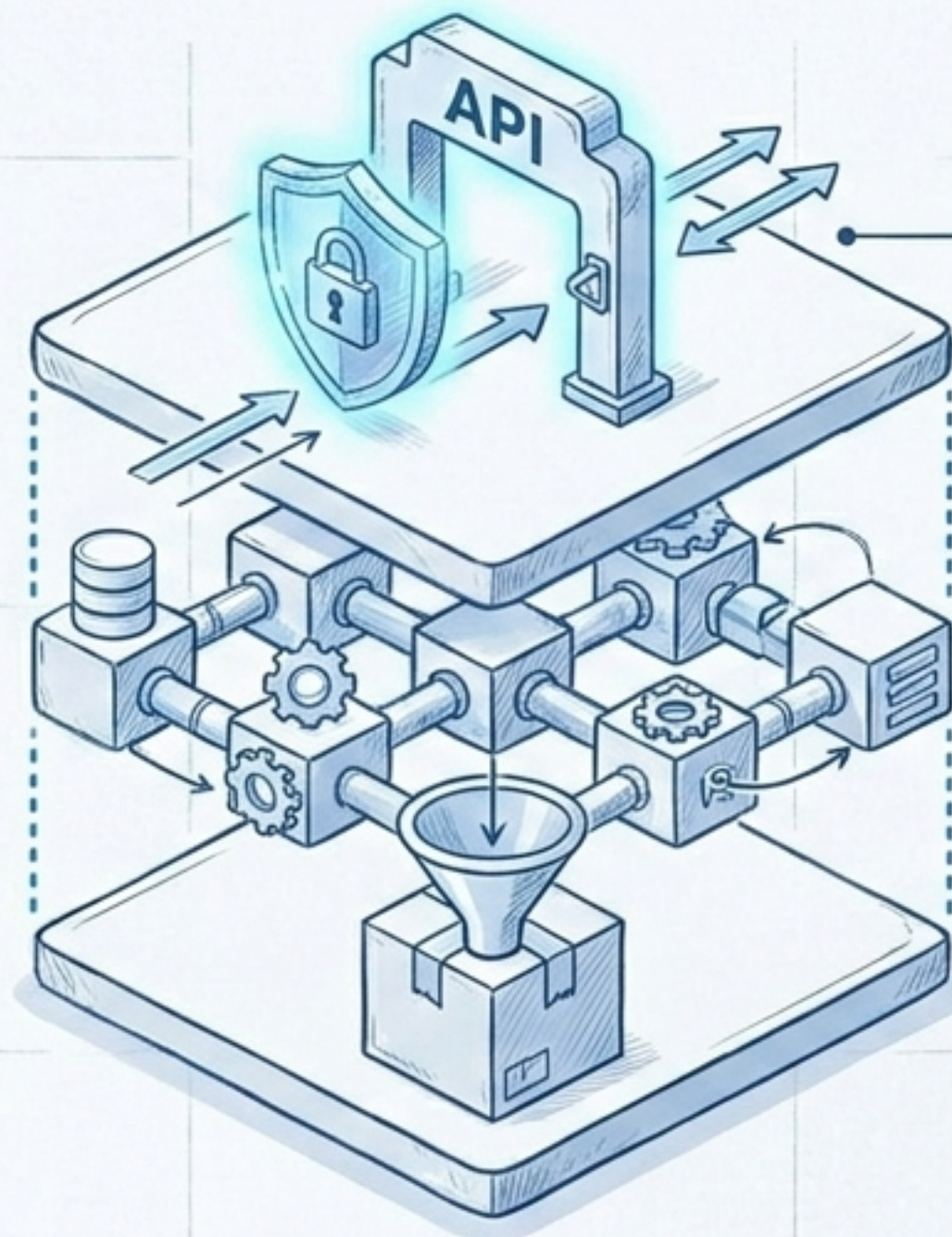


Spring REST Docs

Geração e automação madura de documentação de APIs baseada em testes.



A Anatomia de um Microserviço Contemporâneo



Camada de Exposição

Endpoints RESTful (Spring Web MVC) recebendo chamadas HTTP externas, rigorosamente protegidos pelo Spring Security.

Camada de Mecânica e Lógica

O Spring IoC Container conectando Repositórios (Spring Data) a Controladores de negócio via Injeção de Dependência.

Camada de Infraestrutura e Build

Maven / Gradle gerenciando o empacotamento, plugins e baixando as dependências do Spring Boot.

Do gerenciamento eficiente da build à exposição segura da API, o ecossistema **Spring Framework** é a fundação que viabiliza arquiteturas ágeis, desacopladas e prontas para a **nuvem**.

